

Technical Plan: Data-Driven Generative Visual System (4-Bit Grayscale)

Overview & Influences

The goal is to build a **generative visual system** that **draws from real-world data** (e.g. Home Assistant sensor streams) to create evolving 16-level grayscale imagery. This plan emphasizes a **modular architecture** and **clear logic design**—ensuring that the influence of data on visuals is *meaningful yet tunable by non-experts*. The approach is informed by generative art pioneers who balanced rigid systems with subtle randomness:

- **Vera Molnár** – Worked within strict self-imposed rules (her “machine imaginaire”) and methodically iterated through all possibilities, yet introduced small perturbations (famously “1% disorder”) to inject chance and break perfect order ¹. For example, in *Interruptions* she drew a grid of lines then rotated a few and erased some, creating blanks amid orderly lines ². This taught us that **tiny random deviations can humanize a system** and make outputs more engaging ³. Our system will similarly use data as a source of controlled “disorder” to avoid sterile outputs.
- **Manfred Mohr** – Introduced the concept of using a fixed geometric **structure as an alphabet** for generative exploration. In *Cubic Limit* (1973–75), he took the 12 edges of a cube as base elements and systematically permuted, rotated, and combined them to generate all possible compositions ⁴ ⁵. This demonstrates the power of **constraints + exhaustive variation**. For our project, each visual module may have a predefined structure or motif (grid, particle, waveform, etc.) and the incoming data will select or transform those elements in rule-based ways (ensuring a coherent “alphabet” of forms while still allowing variety).
- **Casey Reas & Processing** – Reas’s *Process* series highlights clarity in generative rules. He defines simple **Forms** (primitives like lines or circles) with **Behaviors** (rules like “move in a line,” “bounce on edges”) to create *Elements* that interact ⁶. The generative artwork emerges from these interactions, producing either **diagrammatic clarity or complex dense patterns** depending on parameter tuning ⁷. Importantly, Reas publishes each work as a short **English description of the rules**, underscoring that **the rules themselves are the art** and the software/hardware are just interpreters ⁸. This inspires us to keep our system’s logic transparent: we can document how sensor data maps to visuals in plain terms, and even allow users to adjust those mappings without delving into code. The influence of Processing (which was created by Reas & Ben Fry to make coding accessible to artists ⁹) encourages us to build our system in an **accessible language/environment** and perhaps provide a simple UI, so that tuning parameters feels like “sketching with code” rather than engineering ¹⁰ ¹¹.
- **Zach Lieberman & Interactive Art** – Lieberman’s work (co-founder of *openFrameworks*) shows how real-time input and creative coding tools can make generative art **playful and accessible** ¹². He often maps physical gestures or daily phenomena to visual outputs in intuitive ways, and he

emphasizes making tools for others (e.g. his open-source toolkit) so non-experts can create with code. In our context, the “gestures” come from sensor data (e.g. the house’s “heartbeat” of energy usage, or the “calm/storm” of environmental changes). We aim to channel Lieberman’s approach by **mapping data changes to visual changes that feel natural**, and by structuring the code in a modular way (like plug-and-play “**visual modules**”, analogous to modules in a creative coding library or a synthesizer) so that others can swap components or adjust settings easily.

With these influences in mind, we proceed to design a system that is **rule-based yet dynamic, constrained (4-bit grayscale)** yet expressive, and above all **legible and tunable**.

System Architecture & Logic Design

We propose a **layered architecture** that cleanly separates concerns. This will make the system easier to understand, modify, and extend:

1. Input Layer (Data Ingestion)

- **Data Sources:** Initially, we will simulate sensor inputs with algorithmic or random values (to focus on the visual logic). For example, one can generate a slow oscillating value to mimic temperature, a random bursty signal to mimic occupancy, or use recorded historical data from Home Assistant. When ready, these will be replaced by **live Home Assistant data** (via its API or an MQTT feed). Home Assistant’s API can provide JSON outputs of sensor states, or it can publish sensor values to an MQTT broker that our system subscribes to. The key is that *all data flows in one direction*: from HA to our system (no control signals sent back) – maintaining a “read-only” ambient observer role.
- **Data Update Mechanism:** A small **data adapter service** will poll or subscribe to updates at a reasonable interval (e.g. once per second or as sensor events arrive). We must manage different update rates – some sensors (temperature) change slowly, others (motion detection) might be instantaneous booleans. The adapter will maintain a dictionary of the latest values for each relevant sensor. For simulation mode, this adapter might simply update each “sensor” with a procedural value (e.g., `temp = 0.5 + 0.5*sin(t)`, `motion = random(0,1) > 0.9 ? 1 : 0`, etc., to simulate day-night cycles, random events, etc.).
- **Normalization:** Raw sensor units vary (degrees, percentage, on/off, etc.). The input layer will normalize values into a **0.0 to 1.0 range** (or -1.0 to 1.0 if needed for bipolar parameters). For example, temperature might map from a plausible min/max range to 0–1; binary sensors become 0 or 1; categorical states could be mapped to an integer or one-hot encoding as needed. This provides a uniform scale for the next layer.

2. Mapping/Interpretation Layer (Data → Semantic Parameters)

This is the **heart of the system’s logic**, where we decide how data influences the visuals. The idea is to derive a set of **semantic channels** (abstract parameters) from the raw data. These semantic parameters are

designed to be intuitively tied to the *character* of the environment, and they drive the visuals. Crucially, this layer is where we allow user **tuning via weights and mappings**:

- **Define Semantic Parameters:** Based on the project brief, we might define a handful of high-level parameters. For example:
 - `activity_level` – representing how “busy” or occupied the environment is. Could be derived from motion sensors count, number of people home, recent movement events, etc.
 - `energy_intensity` – representing power usage or general system “load”. Derived from smart meter data or number of devices running.
 - `environmental_variability` – representing how much the environment is changing (temperature swings, humidity changes, noise fluctuations).
 - `calm_vs_noisy` (or `stability`) – perhaps indicating the steadiness of readings (a very steady state vs. erratic changes).
 - `warmth` or `temperature` – a more literal channel if we want, though we might fold this into an abstract notion like “calm warm evening” vs “cold active day” etc., depending on design metaphor.

These are just examples – a key research step is determining which **metaphors are effective for ambient data display** (question 4.1.3 in the brief). For instance, *weather metaphors* (calm vs stormy visuals) might map well to home activity levels.

- **Mapping Functions:** For each semantic parameter, we define how it’s calculated from one or more sensor values. This can be done with **weighted sums, formulas, or conditional logic**:
 - *Example:* `activity_level = W1 * motion_sensor_count + W2 * door_open_events + W3 * noise_level`. The weights (W1, W2, W3) are tunable. If only motion and door sensors are available, perhaps `activity_level` is a normalized count of triggers in the last X minutes.
 - *Example:* `energy_intensity = normalize(power_usage_watts, min=0, max=5000)` so that 0 kW = 0.0 and 5 kW = 1.0 (with clipping or a nonlinear scale if needed to account for typical range).
 - *Example:* `environmental_variability` could be computed as the rolling **variance** or **derivative** of temperature/humidity readings. E.g., take the last 10 minutes of temperature data and compute how much it has fluctuated (higher variance → more changeable environment).
 - *Example:* `stability` might inversely relate to how many sensors changed state recently – if many changes, stability is low (0), if nothing changed, stability is high (1).

Each mapping can also include transformations: linear maps, thresholding (e.g. if a value exceeds a threshold, cap it), smoothing (using a low-pass filter to avoid jitter in the visuals), or state logic (e.g., detect if house is in “night mode” vs “day mode” via time or light sensors, and feed that as a parameter to possibly switch visual themes).

- **Weights and User Adjustments:** The mapping layer should be **configurable**. A non-expert user might not write code, but they can use a simple UI or config file to adjust weights and contributions. For instance, we could have a JSON or YAML config like:

```
activity_level:
  motion_sensors: 0.7
  door_sensors: 0.3
energy_intensity:
  mains_power_w: 1.0
```

```
variability:
  temperature: 0.5
  humidity: 0.5
```

This means *activity_level* is 70% influenced by motion sensors and 30% by doors; *energy_intensity* directly uses the mains power usage; *variability* takes equal parts temperature and humidity fluctuation. The system would read these weights and apply them. A simple **web interface with sliders** could also be made to adjust these in real-time (sliders for each weight, if the user wants to play with mappings live and see the visual respond differently).

- **One-Way Influence Reminder:** This layer (and the input) operates only in one direction. No matter how extreme the visuals get, they do **not** send any signal back to Home Assistant. This separation simplifies reasoning and ensures the installation is *ambient* (like a mirror reflecting the state) rather than interactive. Conceptually, this lack of feedback loop fosters a contemplative, “*observer*” role for the artwork, as opposed to reactive control systems. It’s akin to Brian Eno’s idea of ambient displays that *reflect* environment mood without trying to alter it.

3. Visual Engine Layer (Generative Modules)

This is where the **graphics are generated**. We design it to be **modular**: multiple interchangeable visual algorithms can run, all adhering to a common interface so they can utilize the semantic parameters from the mapping layer. Key aspects of the visual engine:

- **4-Bit Grayscale Output:** All visuals must render using **16 discrete gray levels (0–15)**. This constraint will shape the aesthetic. We can’t rely on rich color; instead we focus on:
- **Value and Contrast:** Use light vs dark elements to create focus or hierarchy.
- **Texture:** Employ patterns (hatching, stippling, noise) to create the illusion of intermediate tones or just to add visual interest within the 16-tone limit.
- **Dithering:** If a visual effect would naturally demand a gradient or smooth fade beyond 16 levels, implement dithering algorithms. For instance, use **ordered dithering** (like a Bayer matrix) or **error diffusion (Floyd-Steinberg)** to toggle pixels between two nearest gray levels, simulating a in-between tone ¹³ ¹⁴. This could be especially useful if the output device is a higher-res screen; on an actual 4-bit e-ink display, dithering may be built-in for images. Dithering could itself become part of the aesthetic (producing a grainy, dynamic texture as values change).
- **Spatial Resolution & Composition:** Decide on the display resolution (especially on Raspberry Pi, higher resolution means more computing). Possibly start with something like 800×600 or similar in grayscale and see performance. Compositionally, because color isn’t available, composition (placement of elements) and motion become primary ways to keep it engaging.
- **Multiple Visual Modules:** Plan for at least 2–3 distinct visual algorithms (“styles”) that can be swapped. Each module can be thought of like a **plugin** or a mode. Examples of module concepts:
- **Grid Field Module** – Imagine a grid of rectangles or circles that fill the screen. External data influences their sizes, brightness, or arrangement. For instance, `activity_level` could control how many cells are filled or how frequently the grid pattern shifts; `energy_intensity` might control overall brightness (a surge causes the whole grid to glow more). A small randomness

ensures the grid isn't static – perhaps cells flicker or shift with a noise pattern, where the noise intensity is tied to `variability`.

- **Particle Swarm Module** – A number of particles move on screen. They may gravitate toward certain areas or move erratically. Here, `activity_level` could control particle speed or quantity (busy data = faster or more particles on screen), `stability` might influence the motion algorithm (high stability = particles move in smooth, predictable paths; low stability = more jittery or Brownian motion). Particles could leave trails that slowly fade (with only 16 grays to fade through, the fade steps will be discrete but that could give a charming echo effect).
- **Flow Field (Noise) Module** – Use Perlin noise or flow fields to generate swirling patterns (like an abstract ambient wallpaper). Data can perturb the noise field parameters: e.g., `energy_intensity` controls the amplitude of the noise (higher energy makes the pattern more turbulent or high-contrast), `activity_level` could control the flow speed of the field animation (house busy → faster movement in the visuals). This module could create something like an evolving cloudy texture or wave pattern that subtly changes with data.
- **Line-based Plotter-esque Module** – Inspired by Molnár/Mohr, a module that draws lines or geometric forms with slight random offsets. For example, draw a series of concentric squares or circles. Normally they'd be perfectly regular (order), but introduce `1%` to `10%` of perturbation based on an input: e.g., if `variability` is high, randomly skip or misalign a small fraction of lines (imitating Molnár's *1% disorder* concept ¹). This yields a legible structure (the viewer sees a grid or target shape) but with “vibration” or gaps that reflect how dynamic the data is ³. In a calm state, the shape might be almost perfect; in a chaotic state, many lines might be shifted or missing, as if the shape is breaking apart.
- **Module Interface:** To support modularity, define a base class or protocol for visual modules. For example (in a Python-like pseudocode):

```
class VisualModule:
    def __init__(self, width, height):
        # initialize buffers, etc.
        pass
    def update(self, params, dt):
        # params is a dict of semantic parameters (0-1 values)
        # dt is time elapsed since last frame, for animation timing
        pass
    def render(self, surface):
        # draw the current frame onto a given surface/canvas
        # using only 16 gray levels
        pass
```

Each concrete module (GridFieldModule, ParticleModule, etc.) would implement `update` and `render`. The `update()` method uses the incoming `params` to modulate the internal state of the visuals (e.g., adjust particle speeds, add/remove elements, change target positions, etc.), and the `render()` actually draws the frame. This separation allows us to animate things over time (e.g. even if input data is static, particles can still move with last known velocity).

The **main program loop** would then look something like:

```
params = mapping_layer.compute_semantic_params(latest_sensor_data)
current_module.update(params, dt)
current_module.render(screen)
```

where `screen` is a 4-bit grayscale canvas. After rendering, we may need to enforce the 4-bit constraint (if using a typical 8-bit surface, we'd quantize pixel values to 16 levels, unless we draw using a pre-limited palette from the start).

- **Swapping Modules:** The user (or an automated schedule) should be able to switch which visual module is active. This could be manual (select from UI: e.g., a dropdown or buttons for “Grid”, “Particles”, “Flow”) or rule-based (e.g., at night use module A, during day use module B; or if a certain sensor triggers, change style to reflect a special event). Initially, manual switching or a timed rotation (like a “slideshow” of modes) might suffice. Each module is designed to consume the same `params` input structure, so switching doesn't require changing the data pipeline.
- **Performance:** We will keep an eye on efficiency:
 - On the Mac mini (M4), performance will be ample for prototyping, possibly allowing high resolution or more particles.
 - On the Raspberry Pi, we may need to simplify. Using **hardware-accelerated graphics** is key: if we go with Python, libraries like Pygame use SDL (CPU blitting) which might struggle with full-screen animation on Pi. We might leverage OpenGL via PyOpenGL or use a specialized library (e.g., moderngl, or even Processing's Python mode which can use OpenGL).
 - Another approach is to use **openFrameworks (C++)**: openFrameworks is known to run on Raspberry Pi and can handle generative graphics with good performance ¹². It would require writing the modules in C++, but it offers smooth drawing and easy integration with OpenGL shaders if needed. Given Lieberman's involvement in openFrameworks, it aligns well conceptually with our project's lineage.
 - We will start with Python for rapid development. If we hit performance ceilings (e.g., low FPS at desired resolution), we can consider optimizing critical parts (using numpy for pixel operations, Cython for heavy loops, or porting a module to C++/openFrameworks). The modular design means we could even mix: e.g., one module might internally call a small C++ routine or shader for heavy lifting if needed.
 - **Frame Rate & Transitions:** We target maybe 30 FPS on the Pi for smooth motion. If that's not attainable, even 10-15 FPS could be acceptable for an “ambient art” feel (slower, deliberate changes). The visuals need not be frenetic; a slightly lower frame rate with interpolation can appear as gently evolving art rather than a stuttery video.

4. User Interface Layer (Controls & Presets)

Though not mandatory for initial prototyping, designing a simple way for users to **tune and control** the system is important for real-world use:

- **Parameters Exposed:** Likely expose things like:

- The weight values for data mapping (as discussed, sliders for “how much does sensor X influence parameter Y”).
- Switches or dropdown to change the active visual module.
- Possibly high-level “modes” or presets that bundle a particular mapping + module + style (e.g., a preset for “Calm Night” might choose the flow field module with weights that emphasize temperature over motion, producing slow drifting patterns).
- Speed or intensity knobs: e.g., a master “speed” multiplier for animations, a master “contrast” or brightness adjustment if needed (though we mostly want full 0-15 range usage).
- **UI Implementation:** A lightweight method is to run a small web server on the device. The Pi could host a Flask or FastAPI app that serves an HTML page with controls (sliders, buttons) and uses WebSockets or REST calls back to the program to update settings. This way, any phone or laptop can connect to the Pi’s Wi-Fi and adjust settings live. This avoids needing a physical interface.

Another approach (if not real-time adjustable) is simply reading a config file periodically. The user could edit a config (via SSH or a shared network folder) and then the system could watch for changes and reload parameters. This is simpler but less interactive.

- **Preset Management:** Allow saving the current configuration to a file (e.g. a JSON of all weights and current module) and loading it. That way a user can have a few favorite setups (morning vs evening modes, etc.).
- **Analogy to Audio Mixer:** We can explain the UI metaphor in those terms – like mixing different sensor “channels” into the visual output. Each slider is akin to a volume fader for a sensor’s influence, and the visual module is like an effects patch that turns those mixed signals into imagery. This analogy (inspired by the question about using an audio mixing metaphor for visuals) could guide an intuitive design.

Implementation Plan (Phased Development)

To manage complexity, we’ll implement and test the system in stages:

Phase 1: Sandbox Prototyping on Mac (Logic First)

Using the Mac mini (Apple M4 chip) for power and convenience: - **Create a basic graphics window** (using Python with Pygame, or Processing, or openFrameworks C++ project – whichever toolkit we choose). Initially, the window can just display something simple like a blank grayscale gradient or a bouncing shape, to confirm rendering works. - **Implement one example visual module** with dummy inputs to validate the concept. For instance, start with a simple **grid of rectangles** whose brightness oscillates. Simulate one semantic parameter (0–1) that oscillates over time (sine wave or random walk) and map it to something visible (e.g., rectangle size or brightness). This will help get the drawing code right and ensure 4-bit quantization is handled. - **Design the data adapter (simulated):** Create a small thread or loop that updates a set of dummy sensor values. At this stage, just hard-code some evolving values: - e.g., `sim_temp = 0.5 + 0.5*sin(t/60)` (oscillates over a minute), - `sim_motion = (random() < 0.1)` (true occasionally), - etc. Ensure this updates the shared data structure that mapping layer reads. - **Implement Mapping logic:** Write the functions that take the dummy sensor values and compute semantic params. At first, this can be trivial (maybe pass through one value, just to connect the pipeline). Then introduce weighting and

combination: test that adjusting a weight changes the visual outcome (for example, if you have two dummy inputs affecting brightness, tweak their weights in code and see if output changes accordingly). - **Iteration:** Experiment with different visual effects in this sandbox. Try adding noise, try drawing lines, try a small particle system (a list of moving points). This is the creative trial stage, where you use the flexibility of the Mac to see what looks good in 16 shades of gray. Save snippets or configurations that produce interesting results. - **Key Evaluation:** Check legibility and aesthetics. Even with random inputs, do you see distinct *modes* (e.g., when a dummy “activity” goes high, does the image noticeably become more intense or dense)? Adjust mappings as needed to maximize the *perceptibility* of changes – we want the viewer to intuitively sense shifts in the data state.

Phase 2: Integrate Real Data Pipeline

Once the generative logic is solid with simulated data, connect to real Home Assistant data: - **Choose Data Link Method:** E.g., use Home Assistant’s *REST API* to poll sensor states periodically (HA provides endpoints to get the state of an entity). For a more dynamic approach, use the Home Assistant *WebSocket API* which can push updates, or have HA publish MQTT messages for certain topics (sensors). MQTT is often a great choice for IoT streaming; our program can subscribe to relevant topics (e.g., `home/livingroom/temperature`) using an MQTT client library. This provides near-real-time updates with low overhead. - **Data Adapter Implementation:** Replace or augment the simulation thread with a real adapter. It might maintain similar code structure (e.g., on each loop, call HA API or check an MQTT queue). Careful with rate: sensor changes might be sparse, so perhaps poll every 5 seconds for temperature (that’s plenty for slow data) but subscribe for instantaneous events like motion (to update immediately). - **Calibration:** As real data comes in, watch the ranges and behavior. You might find, for example, your normalization for `energy_intensity` was off (maybe your house rarely draws near the 5kW you assumed; you adjust the max down to 3kW = 1.0). Or you realize some sensors update only once per minute, so you may need to interpolate or hold values steady between updates to avoid the visual freezing. - **Testing:** Let the system run for a few hours and record/log the semantic parameters alongside perhaps the visuals. Ensure that when the house gets busy (everyone arrives home, lots of motion, lights on, cooking going on) the semantic parameters indeed spike or change as expected and the visuals respond in a non-trivial way. Also ensure that in a steady period (midnight quiet), the visuals settle into a calmer state. This is where you verify the *legibility*: you want to be able to say “Yes, I can tell it’s calm now because the display is minimal, and I see it went hectic at 6pm when the pattern got very busy”. If not, refine the mapping.

Phase 3: Port to Raspberry Pi & Optimization

With a working system on the Mac (likely using Python or openFrameworks on a desktop), move to the Pi: - **Setup Environment:** Install needed libraries on Pi (Pygame, numpy, etc., or compile openFrameworks project for ARM). Ensure the Pi can access Home Assistant data (network config, possibly use the same MQTT or API endpoints). - **Resolution & Performance Tuning:** Start with a modest resolution and measure frame rate. Profile which parts of the code are slow. Common slow points might be pixel processing in Python; if so, try using vectorized operations (numpy arrays) or offloading some work to the GPU (if using OpenGL). If using openFrameworks, ensure using the Pi GPU drivers (enable GL driver). - **Consider Output Device:** If the plan is to use a special display (like an e-ink screen with 4-bit grayscale), integrate that now. E-ink displays have *very slow refresh* (on the order of seconds for full refresh), which might limit us to very low frame rates but high-quality static images. If that’s the target, the entire strategy might shift to producing “frames” every few seconds rather than continuous animation. (However, given the project description, it sounds like a more conventional screen might be used to allow animation). If using a standard monitor or small HDMI display, just ensure the color depth is honored (we might actually run in 24-bit color but restrict our palette in software). - **Memory:** Watch RAM usage on Pi. If generating large off-screen surfaces or lots

of particles, be mindful to free things or reuse buffers. 16 grayscale could allow us to compress images, but likely not needed if we just draw directly each frame. - **Testing on Pi:** Run through the same scenarios as on Mac. It's crucial to see if timing or multi-threading needs adjustment (the Pi's CPU is weaker, so maybe the data thread and draw thread need to do less work or yield appropriately). Optimize as needed – e.g., reduce number of particles if that was heavy, or precompute any static patterns (like a dither matrix or a base grid) rather than recalculating every frame.

Phase 4: User Interface and Refinement

Now that the core system runs on the Pi with real data, add the polish that makes it usable and adjustable: - **Implement the UI** (if doing a web UI, set up a simple Flask app with slider controls that update global weight variables or switch modules on button press). Test that adjusting sliders indeed changes the visual output in real-time (this will confirm our mapping layer truly reads those weight values each update loop). - **Presets:** Define a few example configs. Perhaps create one preset that highlights *environmental data* (e.g., weight temperature heavily so you get visuals that clearly change day vs night), another that highlights *activity* (weights motion and energy more, giving very busy patterns when the house is active). Save these in a config file so the user can load them easily. You might provide a way to cycle presets (maybe a button or timed schedule). - **Final Aesthetic Tuning:** With everything in place, do a final pass on the aesthetics. This might involve tweaking the drawing styles (maybe the dithering pattern could be adjusted to be less or more obvious, maybe the particle trail length could be tuned, etc.). At this stage, it's useful to get a friend or colleague (ideally someone not deeply familiar with the system) to look at the output and describe what they *think* it represents. If they can say “this looks calm” or “this looks chaotic” and those impressions match the data conditions, then the design is successful. If not, adjust mapping or visuals to better communicate the desired feeling (without making it a literal graph).

Ensuring Balance: Data vs Art

Throughout development, we must keep a critical eye on two often competing aspects: **legibility of data influence** and **artistic integrity**.

- **Avoiding Literal Visualizations:** We deliberately steer away from things like line graphs or numeric readouts. Instead, the data changes the artwork *subtly*. This is akin to Refik Anadol's data-driven paintings, where datasets influence abstract visuals without the viewer needing to know the exact data to appreciate the art ¹⁵. For instance, high energy usage might brighten the whole scene slightly and add more “motion” — the viewer might not know “5000W vs 2000W”, but they'll sense *vibrancy*. Our evaluation criteria include: can a viewer say in general terms “the house seems busy” from the image? If yes, we're conveying data; if not, maybe the mapping is too oblique.
- **Smoothing and Dynamics:** Sensor data can be spiky (e.g., a momentary motion trigger). To prevent sudden jarring visual changes (unless that's desired), we use smoothing in the mapping layer. For example, a sudden spike could be treated as a fast but decaying input (like a strike that then fades) so the visuals can acknowledge it without a single frame flash. This keeps the *ambient* quality – changes in the display should generally be gradual enough to not startle a casual viewer glancing at it.
- **Aesthetic Coherence:** Each visual module should have its own *visual language* (particles vs grid vs flow), but all share the grayscale, and likely some common design principles (e.g., they might all involve repetition and pattern). This gives the overall project a cohesive identity. Even when data is making things “messy”, the output should still look intentionally designed (not just random chaos).

We might borrow Molnár's mindset: *"I love order, but I can't stand too much order"* ³ – meaning we maintain a structured base (order) and add data-driven variation (disorder) in controlled measure.

Tools & Technologies

Summarizing tool choices and libraries based on the above plan:

- **Language:** Python for initial development (for its ease with data handling and rapid prototyping). Use libraries like `pygame` or `processing.py` for graphics. Eventually consider C++ with **openFrameworks** if performance demands (openFrameworks is well-suited for Pi and provides easy drawing functions, and it has been widely used for generative art installations ¹²). Another option is **p5.js** (JavaScript) displayed in a browser, but since we want this standalone on Pi, a native application is preferable.
- **Home Assistant Integration:** Use Python's `requests` or `websocket-client` for API, or `paho-mqtt` for MQTT. Ensure the Pi has network access to HA. Possibly use an **MQTT broker** (HA often includes one) for simplicity: HA sensors publish, our Python subscribes.
- **Data Handling:** `numpy` for efficient numeric operations and smoothing filters (e.g., a rolling average). `asyncio` or threading to handle data fetching without stalling the rendering loop.
- **Graphics:** If sticking with Python, `pygame` can open an SDL window for drawing. It has primitives for drawing shapes, or we can manipulate a NumPy array of pixel values and blit it. For 16-level grayscale, we could use a custom palette or just compute pixel values 0-15 and multiply by e.g. 17 to get 0-255 for display (since $255/15 \approx 17$). If using openFrameworks, it has an `ofApp` with `update()` and `draw()` which map nicely to our structure (we'd just need to pass our `params` to the `ofApp` each frame).
- **Deployment on Pi:** Set the Pi to auto-run the program on boot (so the display starts automatically). Use a lightweight OS or disable desktop if not needed to maximize resources. If using a monitor, match resolution; if using a special display like Pi-compatible e-ink or LED matrix, use the appropriate Python libraries or C++ drivers to draw to those.
- **Version Control & Testing:** Keep the project in Git, test frequently on small changes (especially mapping adjustments – log data vs visuals). Possibly create a mode where the program can record a short video or sequence of frames (to review later or include in documentation).

Conclusion & Next Steps

By following this plan, we will create a system where **external data becomes an artistic "instrument"** – the sensor values are like inputs to a music synthesizer, except they generate visuals. The careful separation of input, mapping, and output ensures we (and users) can understand and tweak the system at each stage:

- We have a clear place to plug in new sensors or data sources (Input Layer).
- We have a transparent mapping that can be explained in simple terms or adjusted live (Mapping Layer).
- We have multiple visual styles to choose from, each of which interprets the data in a unique but adjustable way (Visual Engine Layer).

Throughout development, we'll document our decisions and draw connections back to the research: For instance, if we implement the "line misalignment" visual, we'll note how it was inspired by Molnár's *1% of Disorder* piece ³. If we create a system of moving shapes bouncing and connecting, we can cite Reas's

Form/Behavior grammar as an influence for how simple rules can yield complex results ⁶ . And when ensuring the whole system is configurable by someone other than the coder, we channel the ethos of Processing and openFrameworks – **making generative art accessible and modifiable** ⁹ ¹² .

Finally, as a quick **checklist** going forward: - [] **Grayscale Rendering**: Implement a function to quantize any image or drawing to 16 levels (to enforce the constraint). - [] **Data Simulator**: Module to feed test patterns for rapid iteration. - [] **Semantic Params Defined**: Decide on final set of semantic parameters (aim for 3-5) and document what they mean. - [] **At least 2 Visual Modules** coded and working with the params. - [] **Home Assistant Link** established (with one method, e.g. MQTT) and feeding real data. - [] **Performance on Pi** meets minimum viable frame rate (adjust content accordingly). - [] **Basic UI or Config** in place for weights and module switching. - [] **User Testing**: Observe someone using the interface and viewing the output – gather feedback on clarity and aesthetics.

By iterating through these steps, we will not only build a functioning piece of generative art, but also gain insight into the core research questions about mapping data to visuals, the impact of grayscale limitations, and how to balance **system determinism vs. expressive randomness** in an understandable way. Each decision will be reflected upon in our final documentation, tying the practical implementation back to the rich history of generative art that guided us.

Sources:

- Vera Molnár's method of exploring all combinations under strict rules and adding slight randomness ¹ ² .
- Manfred Mohr's *Cubic Limit* and use of a fixed structure (cube edges) as a generative alphabet ⁴ .
- Casey Reas's *Process* series: rule-based software art with clear mappings from instructions to visuals ⁸ ⁷ .
- The philosophy behind Processing, making code art accessible and bridging design with programming ⁹ ¹¹ .
- Zach Lieberman's interactive art and openFrameworks toolkit, emphasizing creative coding for all ¹² .

¹ Lily R. O'Connell

<https://lrofineart.wordpress.com/>

² ³ Vera Molnár, Pioneer of Computer Art, Dies at 99 | Experimenting with algorithms, she began to employ the principles of computation in her work even before she gained access to an actual computer. | Thaddaeus Ropac

<https://ropac.net/news/1609-vera-molnar-pioneer-of-computer-art-dies-at-experimenting-with-algorithms-she-began-to-employ-the/>

⁴ ⁵ emohr.com

https://www.emohr.com/cubic_limit_1.html

⁶ ⁷ ⁸ Process — REAS.com

<https://reas.com/process>

⁹ ¹⁰ ¹¹ ¹³ ¹⁴ Processing - History of Generative Art — Kate Vass Studio

<https://www.katevassgalerie.com/blog/processing>

12 **Generative Art with Zach Lieberman**

<https://baukunst.co/stories/collabs/generative-art-with-zach-lieberman>

15 **'Painting' with data: how media artist Refik Anadol creates art using ...**

<https://www.wipo.int/en/web/wipo-magazine/articles/painting-with-data-how-media-artist-refik-anadol-creates-art-using-generative-ai-67301>